📸 InstaSphere – Kubernetes-Based Social Media Platform

- Containerized Web Application Deployment using Kubernetes

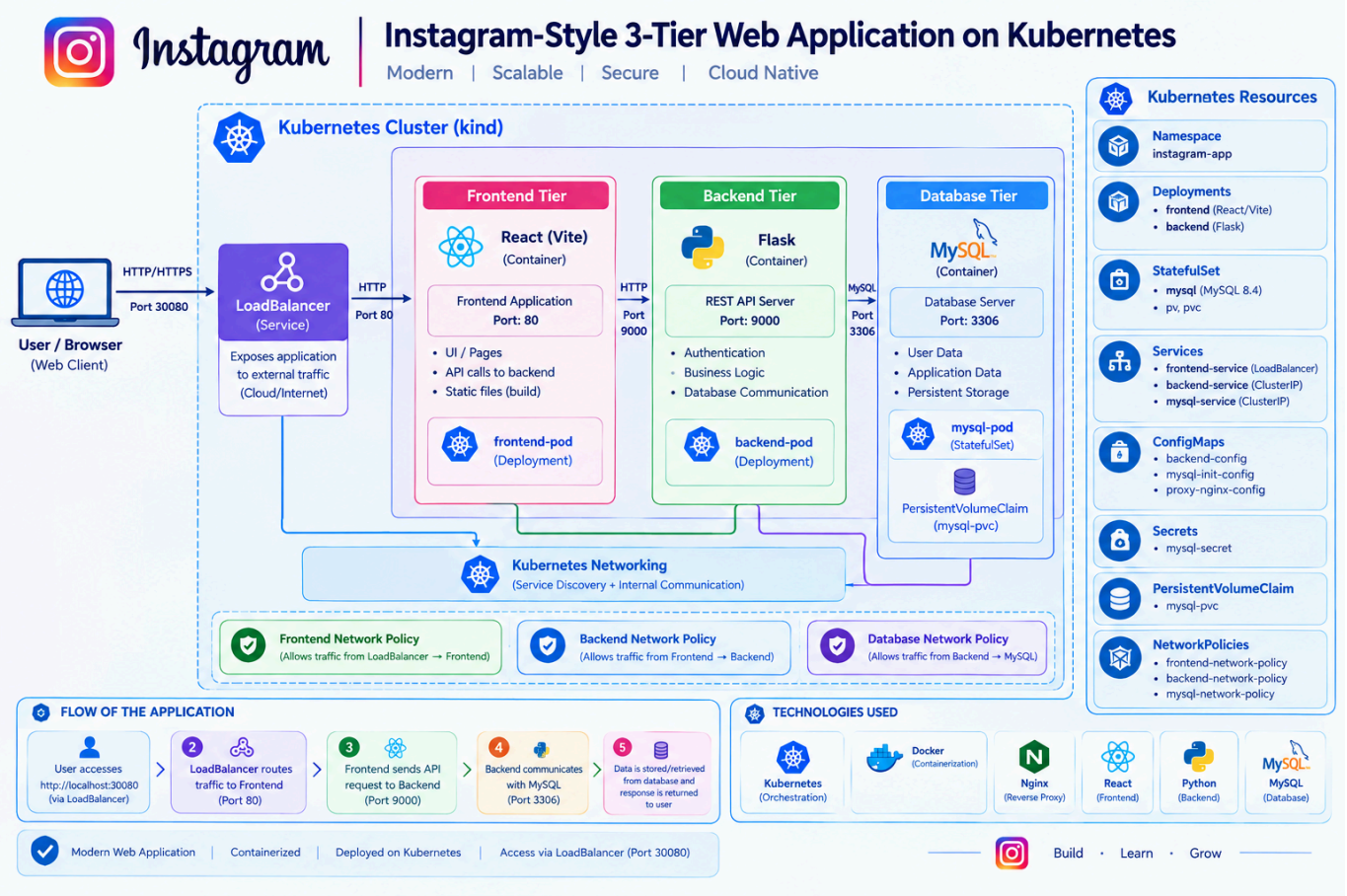
📖 About The Project

This project demonstrates the deployment of a containerized 3-tier web application on Kubernetes.

The application provides an Instagram-style user interface with user registration, login, authentication, and dashboard functionality.

The application is divided into separate frontend, backend, and database tiers. Kubernetes is used to manage application containers, networking, persistent storage, configuration, secrets, and service communication.

🏗️ Architecture Diagram



⚙️ Tech Stack

- Kubernetes
- Docker
- React + Vite
- Nginx
- Python

- Flask
- MySQL 8.4
- Kubernetes LoadBalancer
- ConfigMaps
- Secrets
- PersistentVolumeClaim
- NetworkPolicies
- Kustomize
- YAML

⚡ Project Workflow

```
💻 Application Development
        ↓
🐬 Docker Containerization
        ↓
📦 Build Frontend & Backend Images
        ↓
☁️ Push Images to Docker Hub
        ↓
☸️ Configure Kubernetes Resources
        ↓
⚙️ Deploy Using Kustomize
        ↓
🔍 Verify Kubernetes Resources
        ↓
🔄 Switch NodePort → LoadBalancer
        ↓
🌐 Access Application via LoadBalancer
        ↓
🔐 Register & Login
        ↓
✨ Instagram-Style Dashboard
```

🐬 Step 1 — Building the Application Containers

First, the frontend and backend applications were containerized using Docker.

Frontend

- React + Vite application
- Nginx used to serve the frontend
- Dockerfile created for production build

Backend

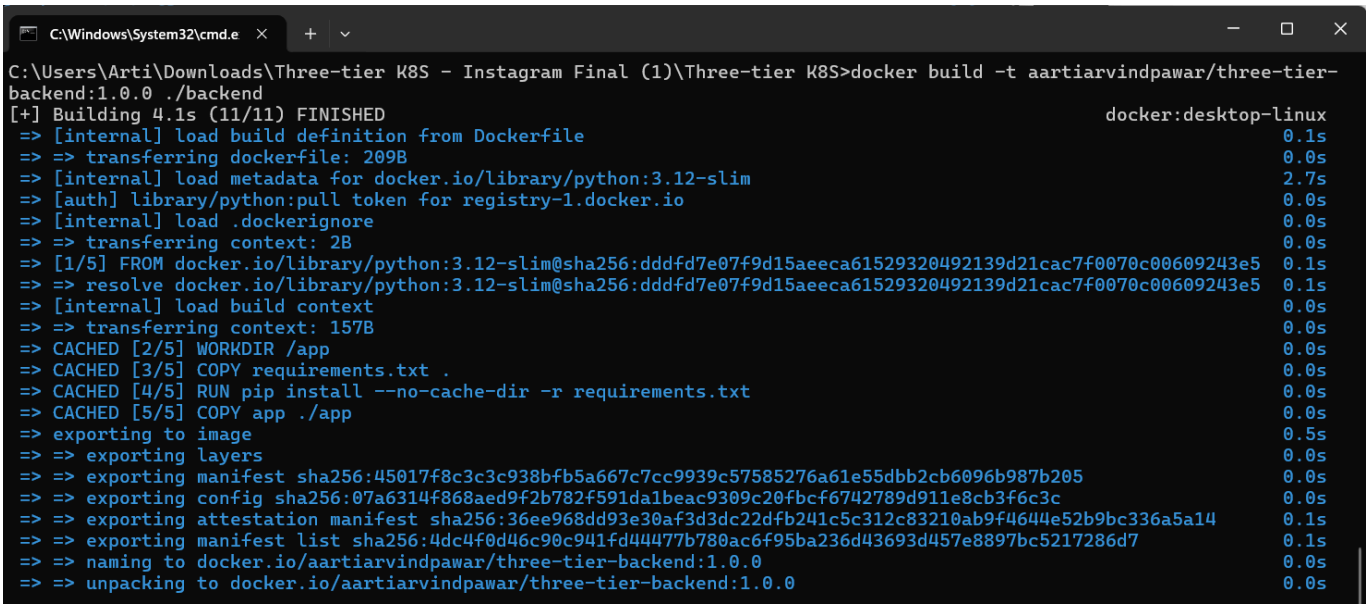
- Python Flask REST API
- Dockerfile created for backend

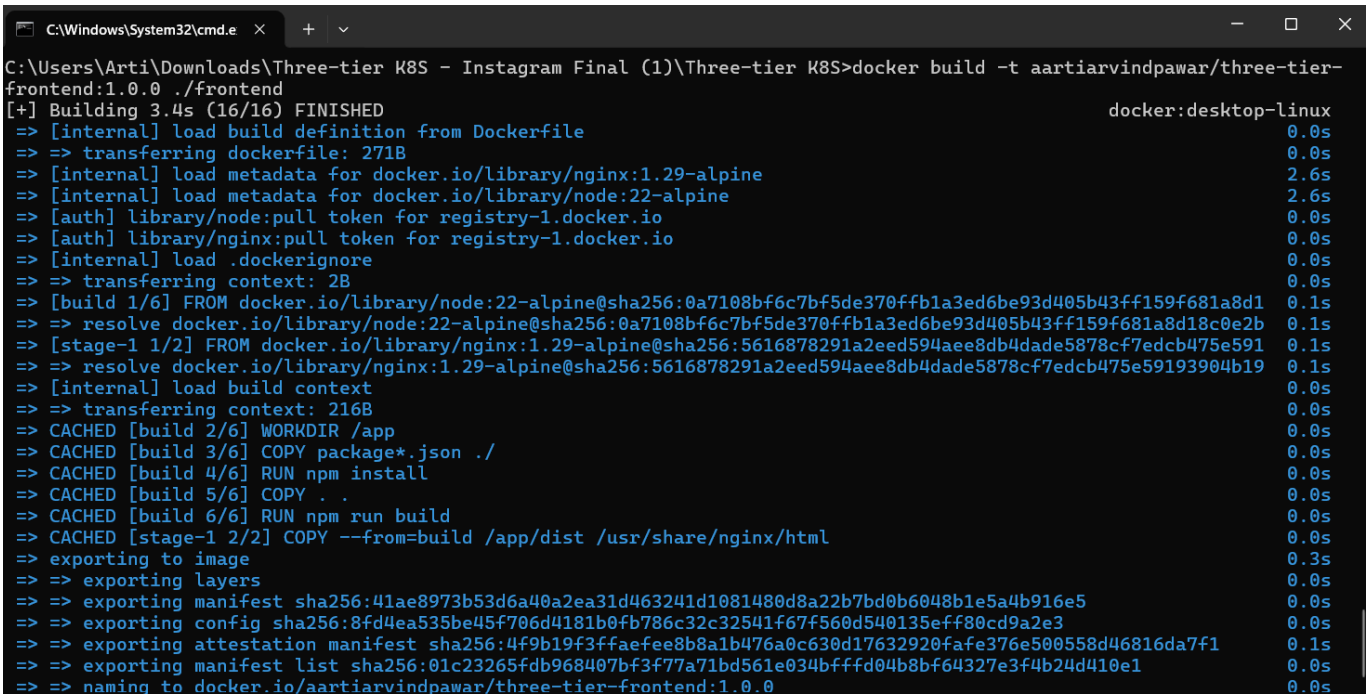
- API handles authentication and database communication

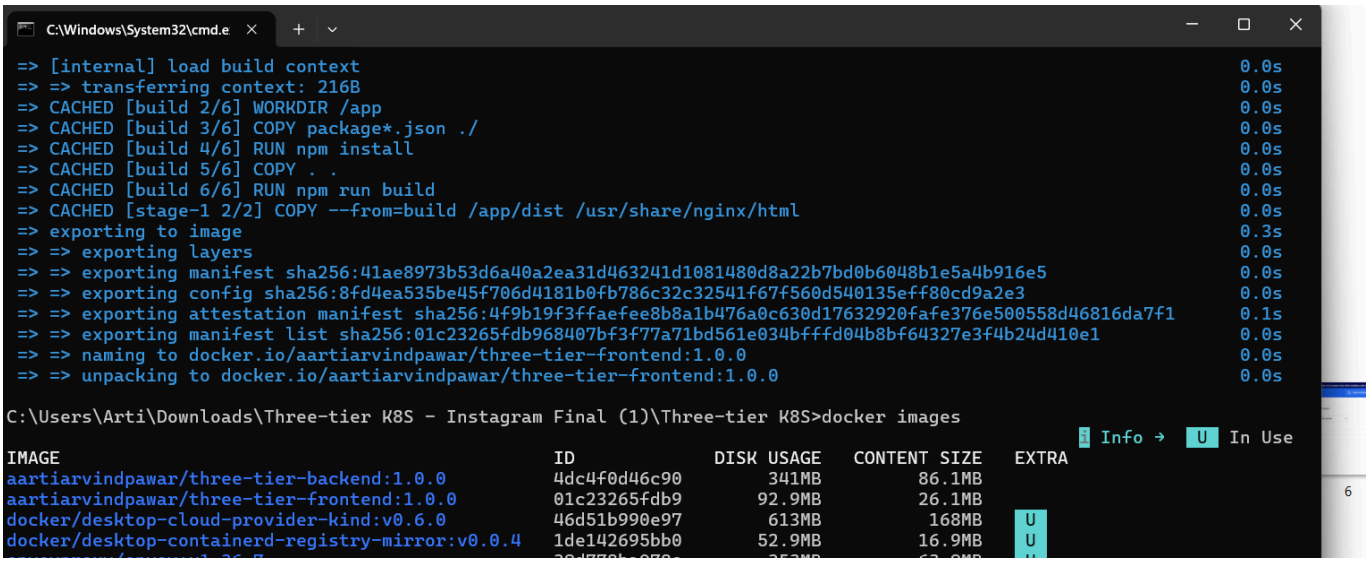
Database

- MySQL 8.4 configured for application data

Built Docker images for application components







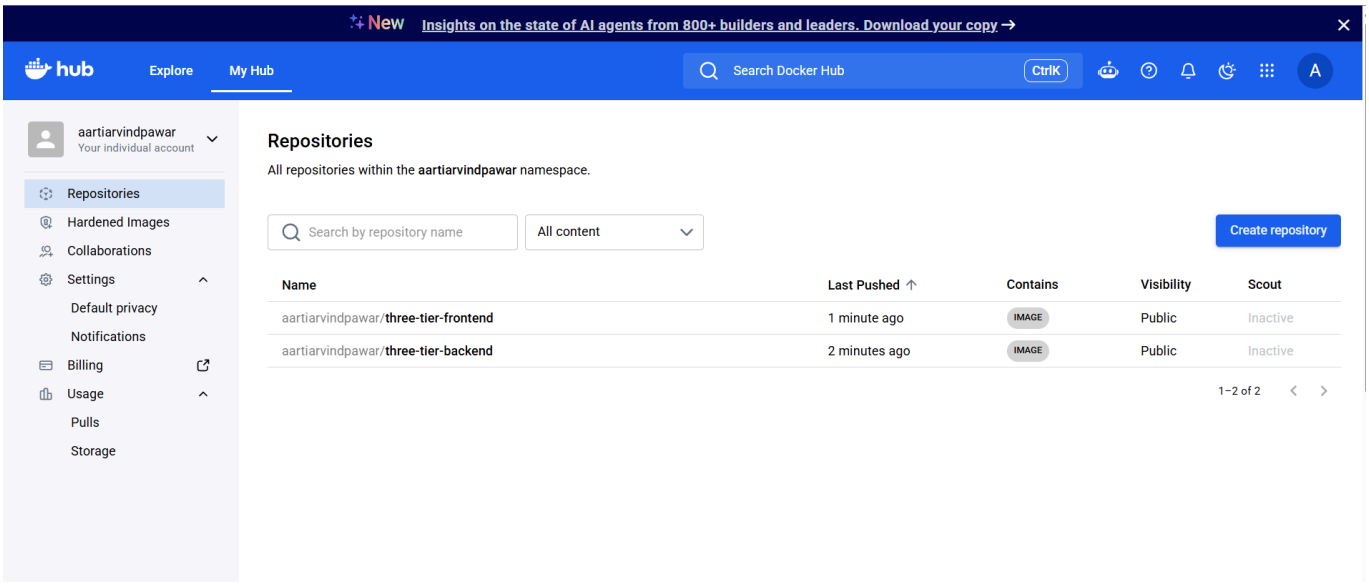
The Docker images were then built successfully for both frontend and backend.

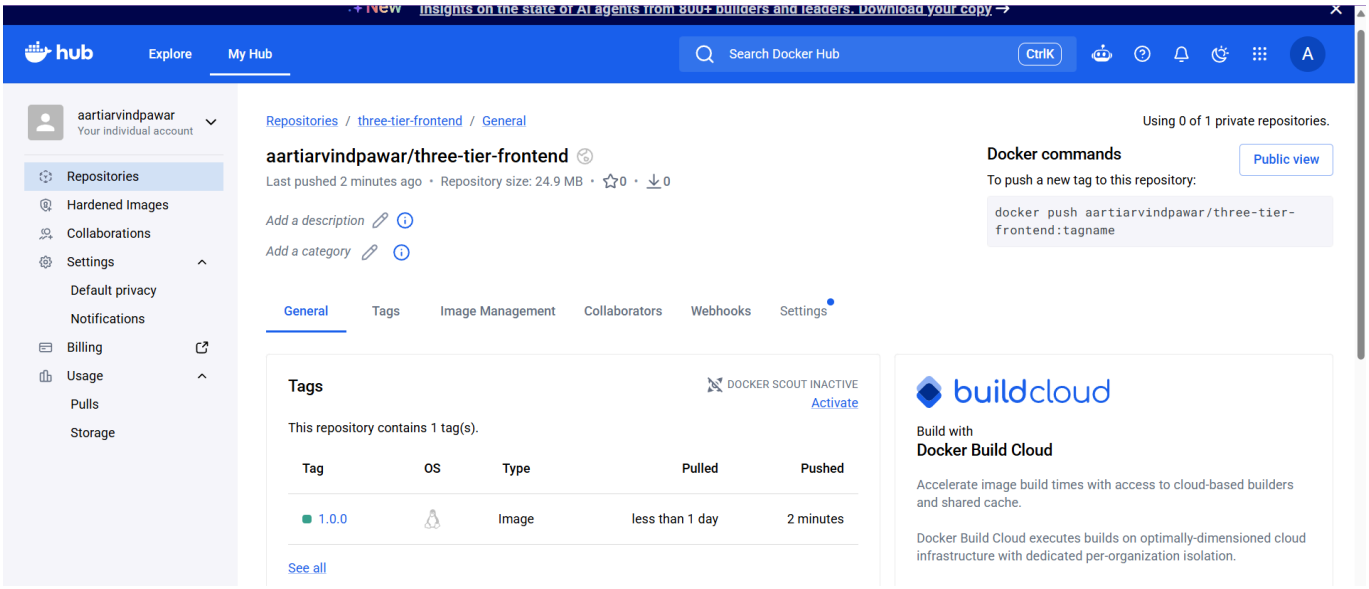
🔹 Step 2 : Push Docker Images

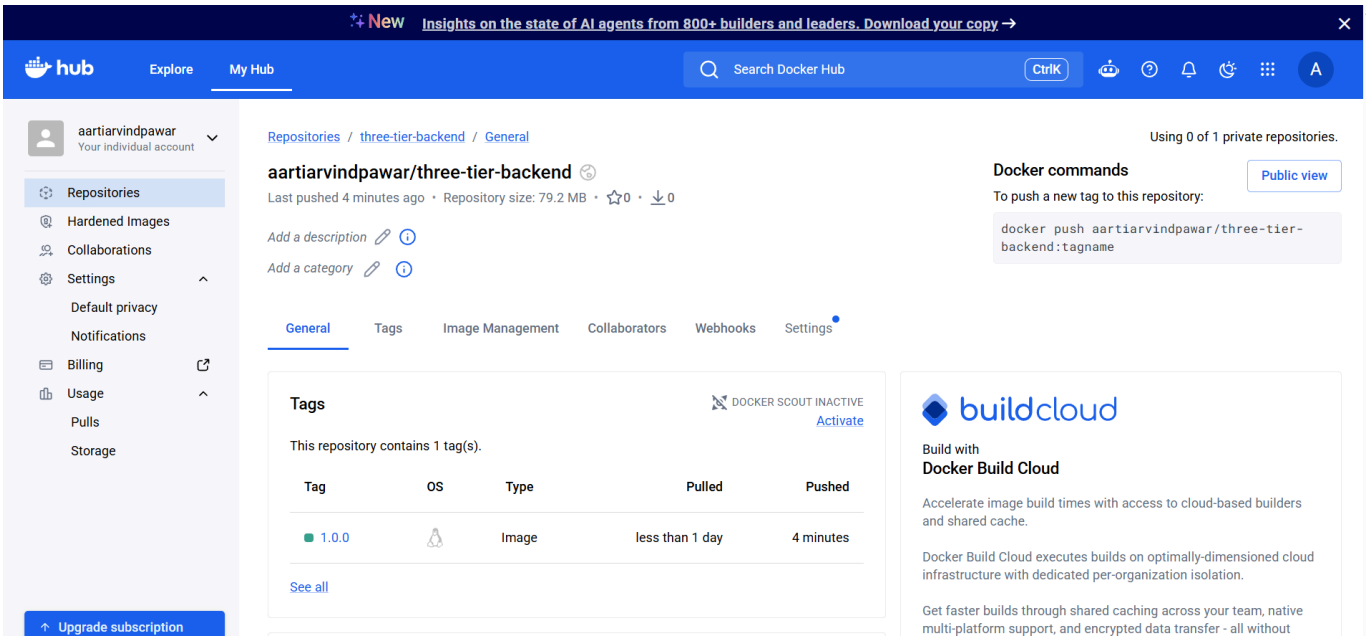
- Tagged Docker images
- Pushed frontend image to Docker Hub
- Pushed backend image to Docker Hub
- Frontend Image:
  - aartiarvindpawar/three-tier-frontend:1.0.0
- Backend Image:
  - aartiarvindpawar/three-tier-backend:1.0.0

```
C:\Users\Arti\Downloads\Three-tier K8S - Instagram Final (1)\Three-tier K8S>docker push aartiarvindpawar/three-tier-backend:1.0.0
The push refers to repository [docker.io/aartiarvindpawar/three-tier-backend]
7d79c21433da: Layer already exists
6b37362b3da7: Layer already exists
318558d103e3: Layer already exists
71e35ead729e: Pushed
bfcd9b17acd5: Layer already exists
e360d85fdc0f: Layer already exists
a510a9ea6d60: Layer already exists
999e300e308d: Layer already exists
5e62c0470a09: Layer already exists
1.0.0: digest: sha256:4dc4f0d46c90c941fd44477b780ac6f95ba236d43693d457e8897bc5217286d7 size: 856

C:\Users\Arti\Downloads\Three-tier K8S - Instagram Final (1)\Three-tier K8S>docker push aartiarvindpawar/three-tier-frontend:1.0.0
The push refers to repository [docker.io/aartiarvindpawar/three-tier-frontend]
aee4e54b3865: Layer already exists
453da7dbc73e: Layer already exists
82736a35d0e7: Layer already exists
781ff50d2644: Layer already exists
583599bb7d38: Layer already exists
4a8b0b2a5b19: Layer already exists
443ff7e57846: Pushed
4be5e3dc456a: Layer already exists
6a0ac1617861: Layer already exists
612c0c1df4c5: Layer already exists
1.0.0: digest: sha256:01c23265fdb968407bf3f77a71bd561e034bfffd04b8bf64327e3f4b24d410e1 size: 856
```





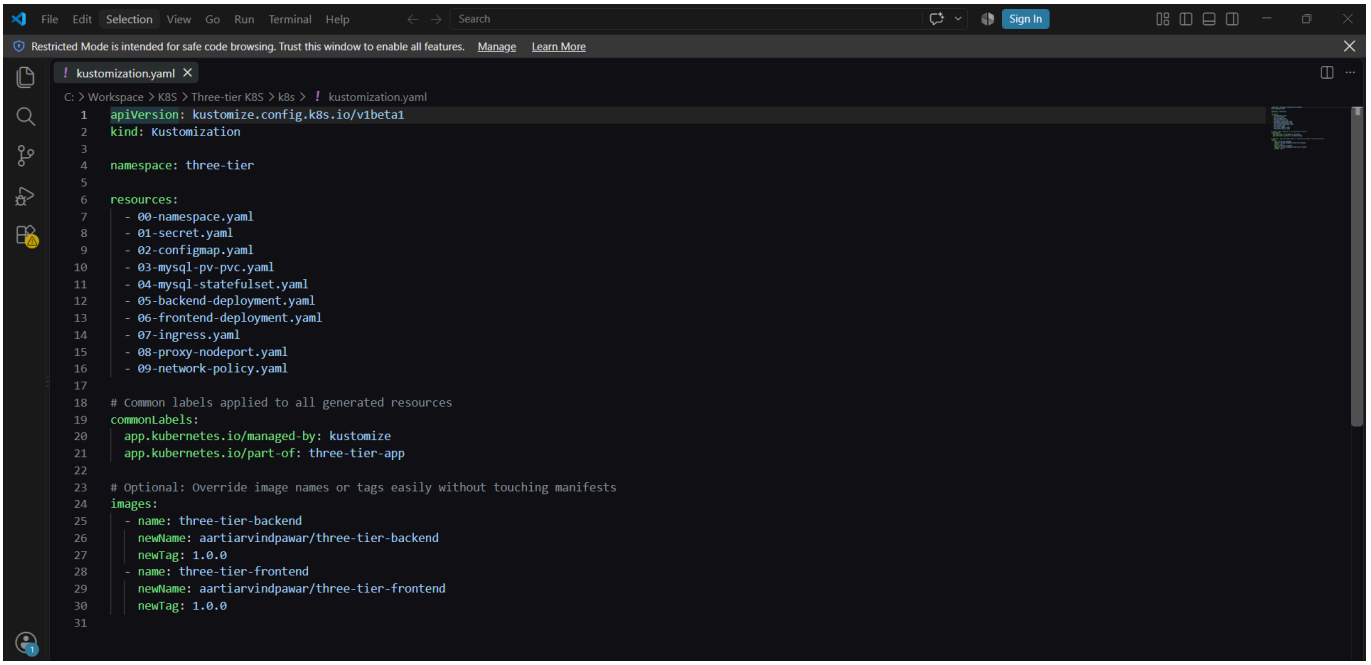


Step 3 — Kubernetes Configuration with Kustomize

The Kubernetes resources were organized using Kustomize. The configuration contains resources for:

- Namespace
- Secrets
- ConfigMaps
- MySQL
- Backend
- Frontend
- Proxy
- Ingress
- NetworkPolicies
- Persistent storage

The project also uses Kustomize to manage image names and versions from a central configuration.



🚀 Step 4 — Deploying the Application on Kubernetes

After preparing the Kubernetes manifests, all resources were deployed using Kustomize.

```
C:\Users\Arti\Downloads\Three-tier K8S - Instagram Final (1)\Three-tier K8S>kubectl apply -k ./k8s/
# Warning: 'commonLabels' is deprecated. Please use 'labels' instead. Run 'kustomize edit fix' to update your Kustomization automatically.
namespace/three-tier unchanged
configmap/backend-config unchanged
configmap/mysql-init-config unchanged
configmap/proxy-nginx-config unchanged
secret/mysql-secret configured
service/backend-service unchanged
service/frontend-service unchanged
service/mysql-service unchanged
persistentvolumeclaim/mysql-pvc unchanged
deployment.apps/backend unchanged
deployment.apps/frontend configured
deployment.apps/proxy unchanged
statefulset.apps/mysql configured
ingress.networking.k8s.io/three-tier-ingress unchanged
networkpolicy.networking.k8s.io/backend-network-policy unchanged
networkpolicy.networking.k8s.io/frontend-network-policy unchanged
networkpolicy.networking.k8s.io/mysql-network-policy unchanged
Error from server: error when applying patch:
{"spec":{"$setElementOrder/ports":[{"port":80}],"ports":[{"name":"http","nodePort":30080,"port":80,"targetPort":80}],"type":"NodePort"}}
to:
Resource: "/v1, Resource=services", GroupVersionKind: "/v1, Kind=Service"
Name: "proxy-service", Namespace: "three-tier"
for: "./k8s/": error when patching "./k8s/": duplicate nodePort: {TCP 30080}

C:\Users\Arti\Downloads\Three-tier K8S - Instagram Final (1)\Three-tier K8S>
```

🔄 Step 5 — Switching from NodePort to LoadBalancer

🔧 From NodePort → LoadBalancer Initially, the application was exposed using NodePort during the earlier setup. For the final implementation, the frontend service was changed to:

- type: LoadBalancer

This change makes the application externally accessible through the Kubernetes LoadBalancer Service instead of relying on NodePort access.

Before

- type: NodePort

Final Configuration

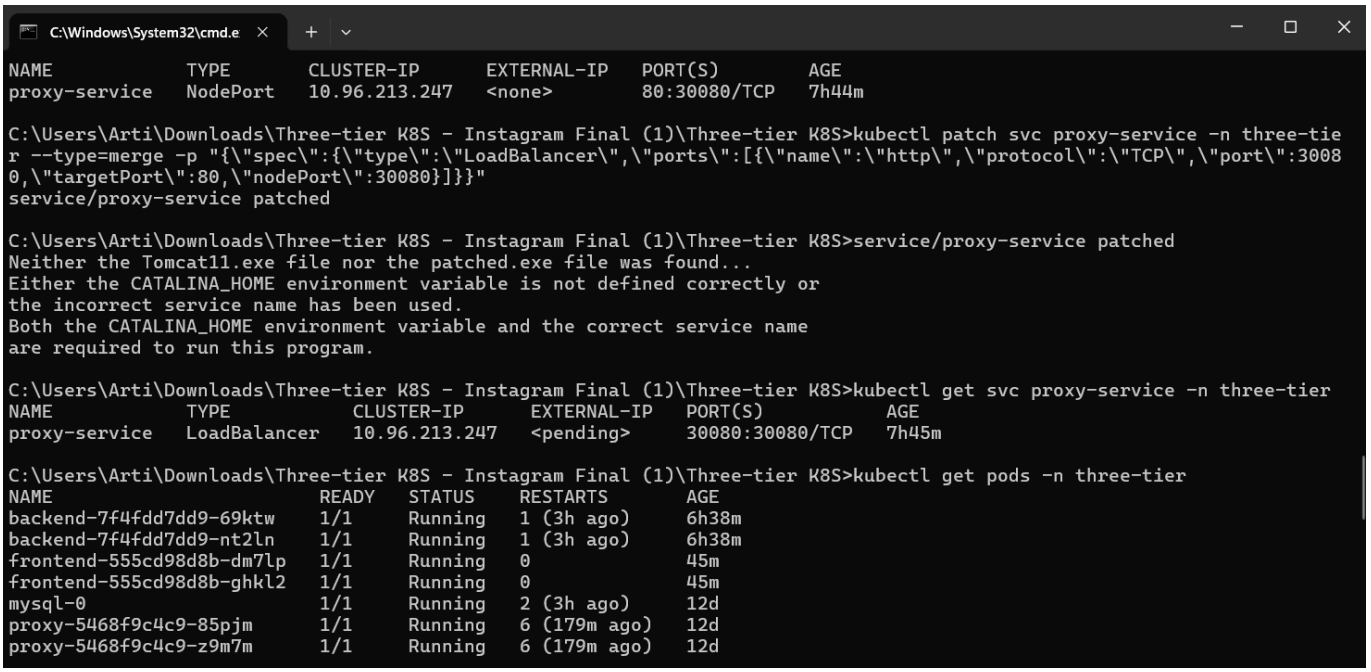
- type: LoadBalancer

After changing the Service configuration, the Kubernetes resources were applied again:

- kubectl apply -k ./k8s/

Then the Service status was verified:

kubectl get svc -n three-tier



🔍 Step 6 — Verifying Kubernetes Resources

Once deployment was completed, all Pods, Services, Deployments and StatefulSets were checked.

- kubectl get all -n three-tier

The verification screenshot shows the application components running, including:

- Backend Pods
- Frontend Pods
- Proxy Pods
- MySQL StatefulSet
- Kubernetes Services
- Deployments

- ReplicaSets

```
C:\Windows\System32\cmd.e
C:\Users\Arti\Downloads\Three-tier K8S - Instagram Final (1)\Three-tier K8S>kubectl get all -n three-tier
NAME                            READY   STATUS    RESTARTS        AGE
pod/backend-7f4fdd7dd9-69ktw    1/1     Running   1 (4h56m ago)   8h
pod/backend-7f4fdd7dd9-nt2ln    1/1     Running   1 (4h56m ago)   8h
pod/frontend-555cd98d8b-dm7lp   1/1     Running   0               162m
pod/frontend-555cd98d8b-ghkl2   1/1     Running   0               162m
pod/mysql-0                     1/1     Running   2 (4h56m ago)   12d
pod/proxy-5468f9c4c9-85pjm      1/1     Running   6 (4h56m ago)   12d
pod/proxy-5468f9c4c9-z9m7m      1/1     Running   6 (4h56m ago)   12d

NAME                       TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)           AGE
service/backend-service    ClusterIP      10.96.228.71    <none>        5000/TCP          12d
service/frontend-service   ClusterIP      10.96.217.83    <none>        80/TCP            12d
service/mysql-service      ClusterIP      10.96.206.57    <none>        3306/TCP          12d
service/proxy-service      LoadBalancer   10.96.213.247   172.18.0.6    30080:30080/TCP   9h

NAME                       READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/backend    2/2     2            2           9h
deployment.apps/frontend   2/2     2            2           9h
deployment.apps/proxy      2/2     2            2           12d

NAME                                  DESIRED   CURRENT   READY   AGE
replicaset.apps/backend-655d44cb6d    0         0         0       9h
replicaset.apps/backend-6b86cc8b6f    0         0         0       9h
replicaset.apps/backend-7f4fdd7dd9    2         2         2       8h
replicaset.apps/frontend-555cd98d8b   2         2         2       162m
replicaset.apps/frontend-65b6f67cfb   0         0         0       9h
replicaset.apps/frontend-6d4bfc46d6   0         0         0       8h
replicaset.apps/frontend-7cc5477bd8   0         0         0       162m
replicaset.apps/frontend-9c5655bd9    0         0         0       9h
replicaset.apps/frontend-ffccd4b64    0         0         0       8h
replicaset.apps/proxy-5468f9c4c9      2         2         2       12d
replicaset.apps/proxy-5654cd4665      0         0         0       12d
replicaset.apps/proxy-5bcfbc8997      0         0         0       12d
replicaset.apps/proxy-5d6dc85df9      0         0         0       12d
replicaset.apps/proxy-7dcb4d5b7d      0         0         0       12d

NAME                     READY   AGE
statefulset.apps/mysql   1/1     12d

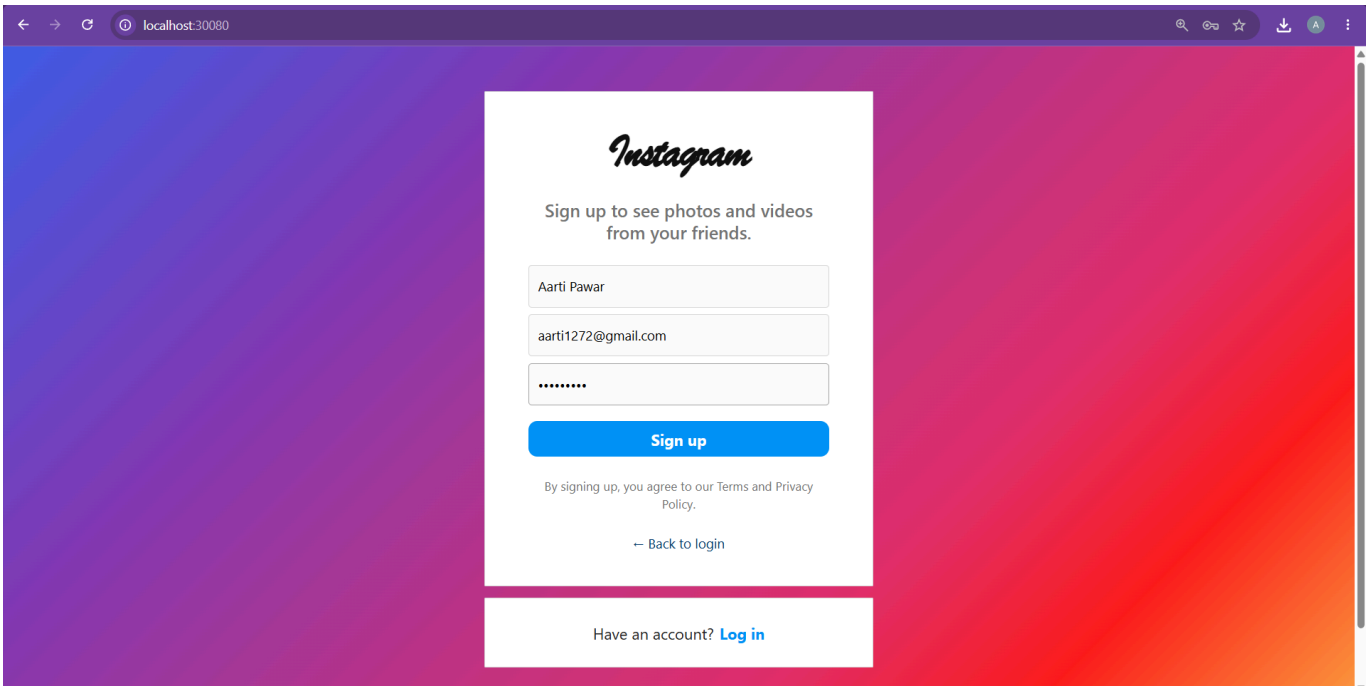
C:\Users\Arti\Downloads\Three-tier K8S - Instagram Final (1)\Three-tier K8S>
```

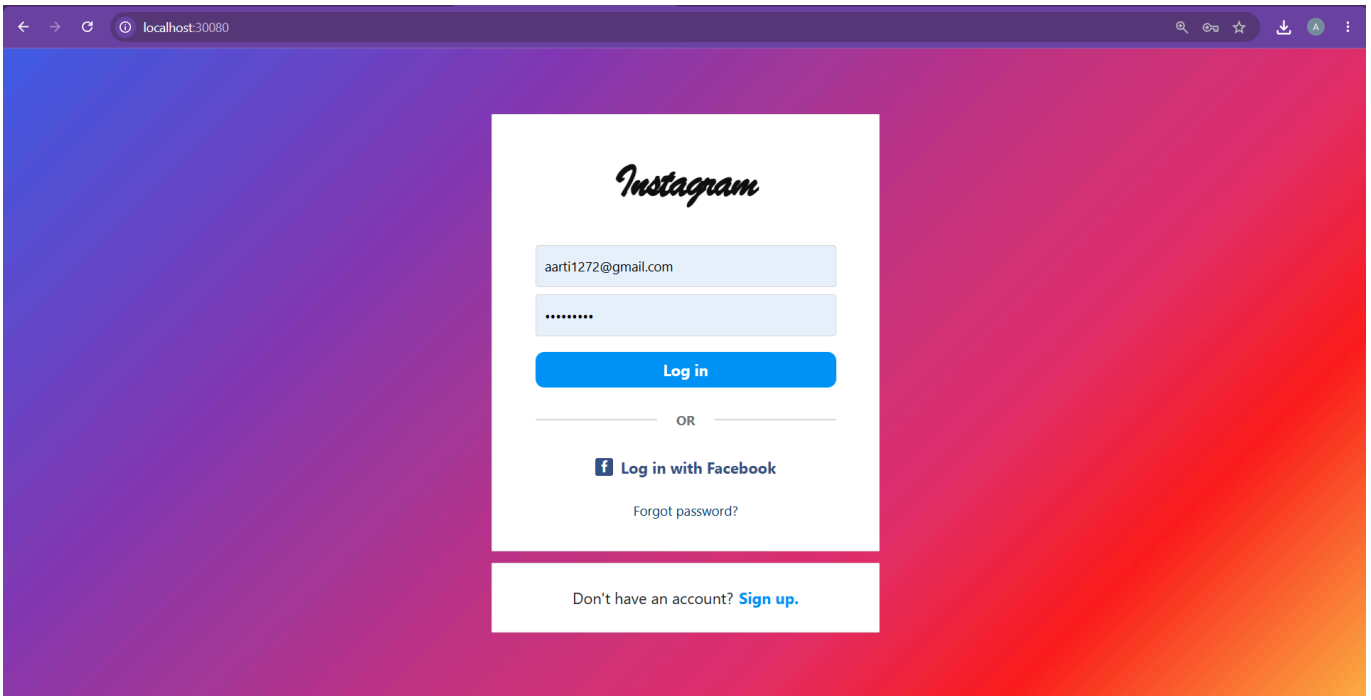
🌐 Step 7 — Accessing the Application

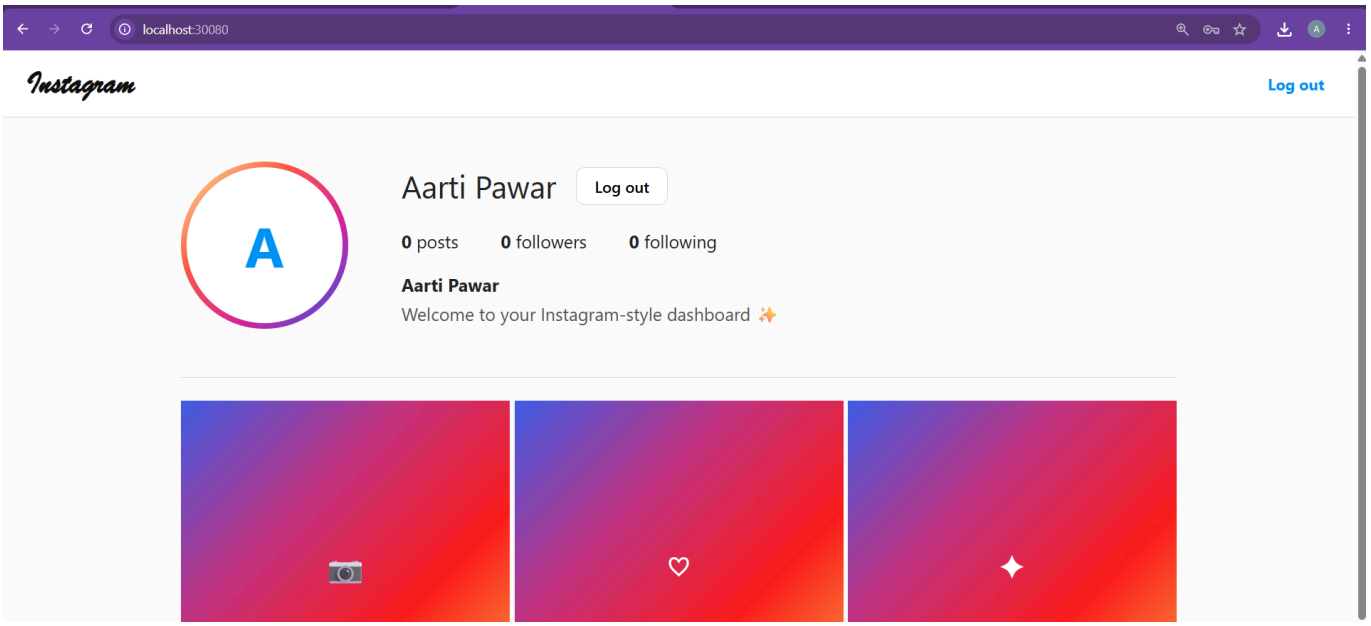
After confirming that the Kubernetes resources were running, the application was accessed through the configured LoadBalancer.

🌍 Application URL

- http://localhost:30080







🏆 Key Highlights

- ✨ Instagram-style user interface
- 🐬 Containerized frontend and backend
- ☸️ Kubernetes-based deployment
- 🔄 NodePort → LoadBalancer migration
- 🌐 External application access through LoadBalancer
- 🔐 Authentication and secure configuration
- 🛡️ NetworkPolicies for controlled communication
- 💾 Persistent MySQL storage
- 📦 Docker Hub image management
- ⚙️ Kustomize-based deployment

💡 What I Learned

Through this project, I gained practical experience in:

- Building Docker images
- Publishing images to Docker Hub
- Deploying applications on Kubernetes
- Working with Deployments and StatefulSets
- Creating Kubernetes Services
- Switching service exposure from NodePort to LoadBalancer
- Configuring persistent storage
- Using ConfigMaps and Secrets
- Implementing NetworkPolicies
- Managing Kubernetes resources using Kustomize
- Verifying application health using kubectl
- Connecting frontend, backend and database components

🎯 Project Outcome

- Successfully deployed an Instagram-style web application on Kubernetes.
- Implemented a containerized 3-tier architecture.
- Configured Kubernetes networking and service discovery.
- Implemented persistent database storage using PVC.
- Secured sensitive configuration using Kubernetes Secrets.
- Controlled inter-tier communication using NetworkPolicies.
- Successfully tested user registration and login functionality.